Gramática Livre de Contexto da linguagem MiniPython

Guia de estudo — BCC328, Construção de Compiladores I

Sumário

1 Para que serve uma GLC neste trabalho **1**

1.1 Definição formal (como na apostila) . . . 1

2 Os terminais: o que o lexer entrega **1**

2.1 INDENT, DEDENT e NEWLINE: as "chaves invisíveis" . . . 2

3 A gramática completa **2**

3.1 Mapa geral . . . 3

3.2 Estrutura do programa e blocos . . . 3

3.3 Comandos simples . . . 4

3.4 Comandos compostos . . . 4

3.5 Funções . . . 5

3.6 Classes . . . 5

3.7 Tipos . . . 6

3.8 Expressões . . . 7

4 Precedência e associatividade: a escada de não terminais **7**

4.1 O problema . . . 7

4.2 A escada (do que liga menos ao que liga mais) . . . 8

4.3 Quem decide a associatividade: o lado da recursão . . . 8

4.4 Precedência em ação: `1 + 2 * 3` . . . 9

4.5 Três detalhes que costumam confundir . . . 9

4.6 Chamadas, índices e campos: *Posfixo* . . . 9

5 Por que não existe o "dangling else" **9**

6 Exemplo trabalhado do início ao fim **10**

7 O que a GLC não consegue dizer **12**

8 Decisões de projeto (e o que você pode mudar) **12**

9 Da GLC aos três parsers **12**

9.1 Descendente recursivo (megaparsec) . . . 13

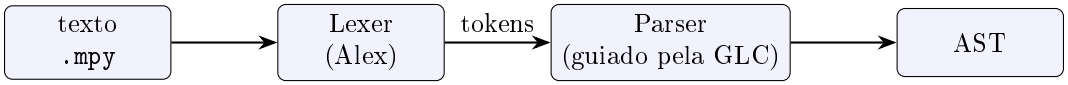
9.2 PEG (typed-peg) 13
9.3 LALR (Happy) 13

10 Exercícios para estudar **14**

11 Resumo de uma página **15**

1 Para que serve uma GLC neste trabalho

O analisador léxico (feito com Alex) transforma o texto do programa em uma lista de *tokens*. Mas uma lista de tokens ainda não diz se o programa está bem formado, nem qual é a sua estrutura. Quem responde isso é o **analisador sintático**, e a **GLC** é a especificação do que ele deve aceitar.



1.1 Definição formal (como na apostila)

Uma gramática livre de contexto é uma tupla $G = (V, \Sigma, R, P)$, em que:

- V é o conjunto de **não terminais** (variáveis). Aqui escrevo em *azul itálico*: *Expr*, *Comando*...
- Σ é o conjunto de **terminais**, que são exatamente os tokens do lexer. Aqui escrevo em `vermelho monoespaçado`: `def`, `+`, `NEWLINE`...
- R é o conjunto de **produções** $A \to \gamma$, regras de reescrita.
- $P \in V$ é o símbolo inicial, aqui *Programa*.

O símbolo λ representa a palavra vazia e | separa alternativas da mesma variável.

Uma sequência de tokens pertence à linguagem de G se existe uma **derivação** $P \Rightarrow \cdots \Rightarrow w$, isto é, uma sequência de passos em que cada passo troca um não terminal pelo lado direito de uma de suas produções. A derivação pode ser desenhada como uma **árvore de análise**, e a **AST** é essa árvore sem os não terminais intermediários e sem a pontuação.

Em palavras simples

A GLC é um conjunto de regras do tipo "um *Comando* pode ser escrito como ...". O parser é o programa que descobre quais regras foram usadas para produzir o seu código. Se nenhuma combinação de regras produz a sequência de tokens, há um erro de sintaxe.

2 Os terminais: o que o lexer entrega

A gramática é escrita sobre os tokens do seu `ExpLexer.x`. A tabela relaciona cada terminal da GLC ao construtor do tipo `Lexeme`.

Terminal na GLC	Token do lexer
`id`	`TkIdent "x"`
`nint`, `nfloat`, `nstr`	`TkIntLit`, `TkFloatLit`, `TkStrLit`
`True`, `False`	`TkBoolLit True/False`
`None`	`TkNoneKw`
`self`	`TkSelf`
`int float bool str`	`TkIntKw TkFloatKw TkBoolKw TkStrKw`
`list`	`TkIdent "list"` (ver nota)
`def class lambda pass`	`TkDef TkClass TkLambda TkPass`
`if elif else while for in`	`TkIf TkElif TkElse TkWhile TkFor TkIn`
`return break continue`	`TkReturn TkBreak TkContinue`
`and or not`	`TkAnd TkOr TkNot`
`+ - * / // % **`	`TkPlus TkMinus TkStar TkSlash TkDSlash TkPercent TkDStar`
`== != < <= > >=`	`TkEqEq TkNeq TkLt TkLe TkGt TkGe`
`=`	`TkAssign`
`+= -= *= /= //= %=`	`TkPlusEq TkMinusEq TkStarEq TkSlashEq TkDSlashEq TkPercentEq`
`( ) [ ] , : . ->`	`TkLParen TkRParen TkLBracket TkRBracket TkComma TkColon TkDot TkArrow`
`NEWLINE INDENT DEDENT EOF`	`TkNewline TkIndent TkDedent TkEOF`

Atenção

Três fatos sobre o lexer que influenciam a gramática:

1. `int`, `float`, `str` e `bool` são palavras reservadas, mas também são usadas como *funções* de conversão: `int(input())`. A gramática precisa aceitá-las como nome de chamada.
2. `list` não é palavra reservada: chega como `TkIdent "list"`. O parser compara a string quando está lendo um tipo, como em `list[int]`.
3. `print`, `input`, `len`, `range` e `append` são identificadores comuns. Para a GLC, `print(x)` é uma chamada de função como qualquer outra.

2.1 INDENT, DEDENT e NEWLINE: as "chaves invisíveis"

Em C, um bloco é `{ ... }`. Em MiniPython, o lexer insere `INDENT` ao entrar em um bloco, `DEDENT` ao sair, e `NEWLINE` ao fim de cada linha lógica. Para a GLC, um bloco é simplesmente

$$\textit{Bloco} \rightarrow \texttt{NEWLINE INDENT } \textit{Cmds} \texttt{ DEDENT}.$$

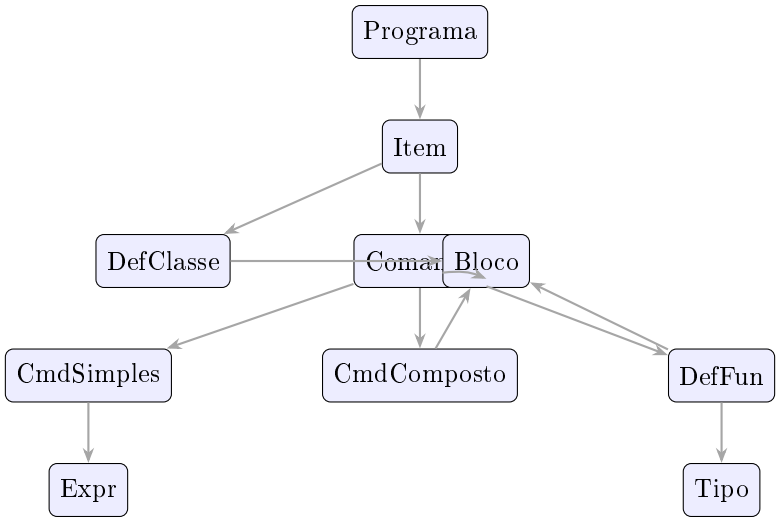
Exemplo de código e dos tokens correspondentes (não aparecem linhas em branco nem comentários):

```
if x > 0:           if id > nint : NEWLINE INDENT
    y = 1               id = nint NEWLINE
else:               DEDENT else : NEWLINE INDENT
    y = 2               id = nint NEWLINE DEDENT
```

3 A gramática completa

Esta é a gramática proposta, dividida em partes. Depois de cada parte há uma explicação do que as regras significam. O símbolo inicial é *Programa*.

3.1 Mapa geral



O programa é uma lista de itens. Cada item é uma classe ou um comando. Comandos incluem definições de função e comandos compostos, e todos eles contêm blocos, que por sua vez contêm comandos. Essa volta é o que dá a recursão da linguagem.

3.2 Estrutura do programa e blocos

$$
\begin{aligned}
\textit{Programa} &\rightarrow \textit{Itens}\ \texttt{EOF} \\
\textit{Itens} &\rightarrow \textit{Item}\ \textit{Itens} \\
&\mid \lambda \\
\textit{Item} &\rightarrow \textit{DefClasse} \\
&\mid \textit{Comando} \\
\textit{Bloco} &\rightarrow \texttt{NEWLINE INDENT}\ \textit{Cmds}\ \texttt{DEDENT} \\
\textit{Cmds} &\rightarrow \textit{Comando} \\
&\mid \textit{Comando}\ \textit{Cmds} \\
\textit{Comando} &\rightarrow \textit{CmdSimples}\ \texttt{NEWLINE} \\
&\mid \textit{CmdComposto} \\
&\mid \textit{DefFun}
\end{aligned}
$$

Em palavras simples

Itens é uma lista que pode ser vazia (a regra com λ encerra a recursão). *Cmds* é uma lista com *pelo menos um* comando, porque um bloco não pode ser vazio (para isso existe o `pass`). Um comando simples termina em `NEWLINE`. Já os comandos compostos e as definições de função não precisam de `NEWLINE` extra, porque o último comando do bloco interno já consumiu o seu. Como *DefFun* está em *Comando*, funções podem ser definidas dentro de outras (funções locais, regra 19 da especificação).

3.3 Comandos simples

CmdSimples → `pass`
| `break`
| `continue`
| `return`
| `return` *Expr*
| `id : ` *Tipo* `=` *Expr*
| *Alvo* `=` *Expr*
| *Alvo* *OpAtr* *Expr*
| *Expr*

Alvo → `id`
| *Posfixo* `[` *Expr* `]`
| *Posfixo* `.` `id`

OpAtr → `+=` | `-=` | `*=` | `/=` | `//=` | `%=`

Em palavras simples

São os comandos de uma linha. Os quatro últimos casos são os mais delicados:

- `n: int = 10` é a declaração com tipo.
- `x = 10`, `xs[i] = 10` e `p.x = 10` são atribuições. O lado esquerdo é um *Alvo*, isto é, algo em que se pode escrever: uma variável, um elemento de lista ou um campo de objeto.
- `i += 1` é a atribuição aumentada.
- `print(x)` ou `p.push(10)` são apenas expressões avaliadas pelo efeito (comando-expressão).

Separar *Alvo* de *Expr* impede que a gramática aceite `f(x) = 2` ou `1 = x`. A regra continua não ambígua, porque nenhuma *Expr* pode vir seguida de `=`.

3.4 Comandos compostos

CmdComposto → `if` *Expr* `:` *Bloco* *Elifs* *Else*
| `while` *Expr* `:` *Bloco*
| `for id in` *Expr* `:` *Bloco*

Elifs → `elif` *Expr* `:` *Bloco* *Elifs*
| λ

Else → `else :` *Bloco*
| λ

Em palavras simples

O `if` tem uma condição, um bloco, zero ou mais `elif` e um `else` opcional. O `for` usa uma expressão qualquer depois de `in`. O caso `for i in range(10)` não precisa de regra especial, porque `range(10)` é só uma chamada de função. Um `elif` sozinho é erro de sintaxe de graça: nenhuma regra de *Comando* começa com `elif`.

3.5 Funções

$$
\begin{aligned}
\textit{DefFun} &\rightarrow \texttt{def id (}\ \textit{Params}\ \texttt{)}\ \textit{Retorno}\ \texttt{:}\ \textit{Bloco} \\
\textit{Params} &\rightarrow \textit{ListaParam} \\
&\mid \lambda \\
\textit{ListaParam} &\rightarrow \textit{Param} \\
&\mid \textit{ListaParam}\ \texttt{,}\ \textit{Param} \\
\textit{Param} &\rightarrow \texttt{id} \\
&\mid \texttt{id :}\ \textit{Tipo} \\
\textit{Retorno} &\rightarrow \texttt{->}\ \textit{Tipo} \\
&\mid \lambda
\end{aligned}
$$

Em palavras simples

Os tipos dos parâmetros e do retorno são *opcionais*, porque a especificação permite inferência (`def identidade(x): return x`). É por isso que *Param* tem duas formas e *Retorno* pode ser λ. A lista de parâmetros é recursiva à esquerda, como a de soma na apostila (*ListaParam* → *ListaParam* `,` *Param*), e no parser vira um `sepBy1`.

3.6 Classes

$$
\begin{aligned}
\textit{DefClasse} &\rightarrow \texttt{class id}\ \textit{Heranca}\ \texttt{:}\ \texttt{NEWLINE INDENT}\ \textit{CorpoClasse}\ \texttt{DEDENT} \\
\textit{Heranca} &\rightarrow \texttt{(}\ \texttt{id}\ \texttt{)} \\
&\mid \lambda \\
\textit{CorpoClasse} &\rightarrow \textit{Campos}\ \textit{Metodos} \\
\textit{Campos} &\rightarrow \textit{Campo}\ \textit{Campos} \\
&\mid \lambda \\
\textit{Campo} &\rightarrow \texttt{id :}\ \textit{Tipo}\ \texttt{NEWLINE} \\
\textit{Metodos} &\rightarrow \textit{Metodo}\ \textit{Metodos} \\
&\mid \lambda \\
\textit{Metodo} &\rightarrow \texttt{def id (}\ \texttt{self}\ \textit{ParamsM}\ \texttt{)}\ \textit{Retorno}\ \texttt{:}\ \textit{Bloco} \\
\textit{ParamsM} &\rightarrow \texttt{,}\ \textit{ListaParam} \\
&\mid \lambda
\end{aligned}
$$

Em palavras simples

Uma classe tem um nome, uma superclasse opcional (herança simples) e um corpo em que **primeiro vêm os campos, depois os métodos**. Todo método recebe `self` como primeiro parâmetro. Como `self` é um token próprio (`TkSelf`) e não um `TkIdent`, ele aparece explicitamente na regra de *Metodo*. A restrição "campos antes de métodos" é imposta pela própria sintaxe, pela ordem *Campos Metodos*.

3.7 Tipos

$$
\begin{aligned}
\textit{Tipo} &\rightarrow \texttt{int} \mid \texttt{float} \mid \texttt{bool} \mid \texttt{str} \mid \texttt{None} \\
&\mid \texttt{id} \\
&\mid \texttt{list [} \textit{Tipo} \texttt{]} \\
&\mid \texttt{(} \textit{ListaTipoOpt} \texttt{) -> } \textit{Tipo} \\
\textit{ListaTipoOpt} &\rightarrow \textit{ListaTipo} \\
&\mid \lambda \\
\textit{ListaTipo} &\rightarrow \textit{Tipo} \\
&\mid \textit{ListaTipo} \texttt{ , } \textit{Tipo}
\end{aligned}
$$

Em palavras simples

Um tipo é um tipo primitivo, um nome (de classe, como `Ponto`, ou de variável de tipo, como `a`), uma lista de algum tipo, ou um tipo de função como `(int, int) -> bool`. O tipo de função é recursivo à direita, então `(int) -> (int) -> int` lê-se como `(int) -> ((int) -> int)`. Sintaticamente, `Ponto` e `a` são ambos um `id`. Quem diferencia maiúscula de minúscula é a fase semântica.

3.8 Expressões

$$
\begin{aligned}
\textit{Expr} &\to \texttt{lambda}\ \textit{ParamsL}\ \texttt{:}\ \textit{Expr} \\
&\mid \textit{Cond} \\
\textit{ParamsL} &\to \textit{ListaId} \mid \lambda \\
\textit{ListaId} &\to \texttt{id} \mid \textit{ListaId}\ \texttt{,}\ \texttt{id} \\
\textit{Cond} &\to \textit{Disj}\ \texttt{if}\ \textit{Disj}\ \texttt{else}\ \textit{Expr} \\
&\mid \textit{Disj} \\
\textit{Disj} &\to \textit{Disj}\ \texttt{or}\ \textit{Conj} \mid \textit{Conj} \\
\textit{Conj} &\to \textit{Conj}\ \texttt{and}\ \textit{Neg} \mid \textit{Neg} \\
\textit{Neg} &\to \texttt{not}\ \textit{Neg} \mid \textit{Comp} \\
\textit{Comp} &\to \textit{Soma}\ \textit{OpRel}\ \textit{Soma} \mid \textit{Soma} \\
\textit{OpRel} &\to \texttt{==} \mid \texttt{!=} \mid \texttt{<} \mid \texttt{<=} \mid \texttt{>} \mid \texttt{>=} \\
\textit{Soma} &\to \textit{Soma}\ \texttt{+}\ \textit{Termo} \\
&\mid \textit{Soma}\ \texttt{-}\ \textit{Termo} \\
&\mid \textit{Termo} \\
\textit{Termo} &\to \textit{Termo}\ \texttt{*}\ \textit{Pot} \\
&\mid \textit{Termo}\ \texttt{/}\ \textit{Pot} \\
&\mid \textit{Termo}\ \texttt{//}\ \textit{Pot} \\
&\mid \textit{Termo}\ \texttt{\%}\ \textit{Pot} \\
&\mid \textit{Pot} \\
\textit{Pot} &\to \textit{Unario}\ \texttt{**}\ \textit{Pot} \mid \textit{Unario} \\
\textit{Unario} &\to \texttt{-}\ \textit{Unario} \mid \textit{Posfixo} \\
\textit{Posfixo} &\to \textit{Posfixo}\ \texttt{(}\ \textit{Args}\ \texttt{)} \\
&\mid \textit{Posfixo}\ \texttt{[}\ \textit{Expr}\ \texttt{]} \\
&\mid \textit{Posfixo}\ \texttt{.}\ \texttt{id} \\
&\mid \textit{Conv}\ \texttt{(}\ \textit{Args}\ \texttt{)} \\
&\mid \textit{Primario} \\
\textit{Conv} &\to \texttt{int} \mid \texttt{float} \mid \texttt{str} \mid \texttt{bool} \\
\textit{Primario} &\to \texttt{nint} \mid \texttt{nfloat} \mid \texttt{nstr} \mid \texttt{True} \mid \texttt{False} \mid \texttt{None} \\
&\mid \texttt{id} \mid \texttt{self} \\
&\mid \texttt{(}\ \textit{Expr}\ \texttt{)} \\
&\mid \texttt{[}\ \textit{Args}\ \texttt{]} \\
\textit{Args} &\to \textit{ListaExpr} \mid \lambda \\
\textit{ListaExpr} &\to \textit{Expr} \mid \textit{ListaExpr}\ \texttt{,}\ \textit{Expr}
\end{aligned}
$$

A próxima seção explica a lógica dessa escada de não terminais, que é a parte mais importante da gramática.

4 Precedência e associatividade: a escada de não terminais

4.1 O problema

A gramática mais simples para somas e produtos é $E \to E + E \mid E * E \mid n$. Ela é **ambígua**: a entrada $1 + 2 * 3$ tem duas árvores, e uma delas calcula $(1 + 2) * 3$. A apostila resolve isso criando

um não terminal por nível de precedência:

$$E \to E + T \mid T \qquad T \to T * F \mid F \qquad F \to n \mid (E).$$

O *Expr* do MiniPython é a mesma ideia com mais degraus.

4.2 A escada (do que liga menos ao que liga mais)

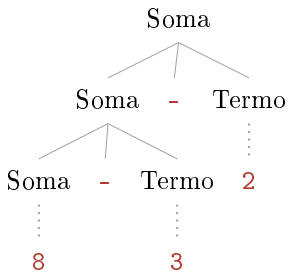
Nível	Não terminal	Operadores	Associatividade
1	*Expr*	`lambda`	(corpo é uma *Expr*)
2	*Cond*	`e1 if c else e2`	direita
3	*Disj*	`or`	esquerda
4	*Conj*	`and`	esquerda
5	*Neg*	`not`	prefixo
6	*Comp*	`== != < <= > >=`	*não associativa*
7	*Soma*	`+ -`	esquerda
8	*Termo*	`* / // %`	esquerda
9	*Pot*	`**`	direita
10	*Unario*	`-` (unário)	prefixo
11	*Posfixo*	`f(...)` `x[...]` `x.campo`	esquerda
12	*Primario*	literais, `id`, `self`, `( ...)`, `[ ...]`	—

Em palavras simples

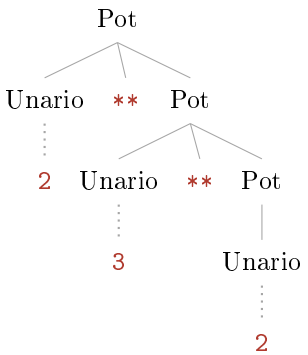
Regra de ouro: um nível só pode usar operandos do nível logo abaixo. Por exemplo, *Soma* tem operandos *Termo*. Assim, um `*` nunca pode ficar "acima" de um `+` na árvore, a menos que você use parênteses, e os parênteses são o caminho de volta à *Expr* (*Primario* → `( Expr )`). Quanto mais para baixo o nível, mais "grudados" ficam os operandos, e por isso o *Primario* é o mais forte de todos.

4.3 Quem decide a associatividade: o lado da recursão

- **Recursão à esquerda**, como em *Soma* → *Soma* `-` *Termo*: o operador agrupa à esquerda, então `8 - 3 - 2` vale $(8-3)-2=3$.
- **Recursão à direita**, como em *Pot* → *Unario* `**` *Pot*: o operador agrupa à direita, então `2 ** 3 ** 2` vale $2^{(3^2)}=512$.



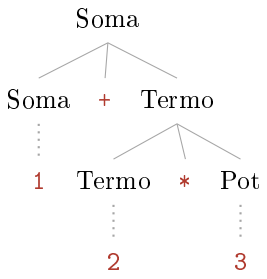
`8 - 3 - 2`: aninha à esquerda



`2 ** 3 ** 2`: aninha à direita

As arestas pontilhadas abreviam a cadeia de não terminais intermediários (por exemplo *Termo* → *Pot* → ⋯ → *Primario*) até o literal.

4.4 Precedência em ação: `1 + 2 * 3`



O `*` está mais fundo na árvore, então é calculado primeiro: $1 + (2 \cdot 3) = 7$. Essa é a única árvore possível, logo não há ambiguidade.

4.5 Três detalhes que costumam confundir

Decisão de projeto

1. O `-` unário liga mais forte que `**`. A especificação (§6.3) diz que a precedência é "`-` unário > `**` > `*` > `+`". Por isso *Unario* está abaixo de *Pot*, e `-2 ** 2` vale $(-2)^2 = 4$. No Python seria -4; aqui seguimos a especificação da disciplina. Mesmo assim `2 ** -1` é aceito, porque o operando direito de `**` é uma *Pot*, que desce até *Unario*.
2. Comparações não encadeiam. A regra *Comp* → *Soma OpRel Soma* não tem recursão, então `a < b < c` é erro de sintaxe. A especificação diz que comparam-se "dois valores do mesmo tipo", e `(a < b) < c` seria `bool < int`.
3. `not` fica abaixo das comparações. `not x > 3` é `not (x > 3)`, o mesmo que no Python, e `and` liga mais forte que `or`, como a especificação pede.

4.6 Chamadas, índices e campos: *Posfixo*

Posfixo é recursivo à esquerda, então ele aceita encadeamentos de qualquer tamanho:

```
self.dados[i]   p.pop()   f(x)(y)   Retangulo.__init__(self, a, b)
```

Cada sufixo (`(...)`, `[...]`, `.id`) é aplicado ao que veio antes, da esquerda para a direita. A chamada qualificada `Pai.__init__(self, ...)` não precisa de regra extra: é um *Primario* `id`, seguido de `.id`, seguido de uma chamada. A regra *Conv* `(` *Args* `)` existe só por causa das palavras reservadas `int`, `float`, `str` e `bool`, que no lexer não são `TkIdent`.

5 Por que não existe o "dangling else"

Na apostila, `S` → `if E then S` | `if E then S else S` é ambígua: em `if a then if b then x else y`, a qual `if` pertence o `else`? No MiniPython o problema desaparece, porque cada bloco é fechado por um `DEDENT`:

```
if a:                       if a :  NEWLINE INDENT
    if b:                       if b :  NEWLINE INDENT
        x = 1                       x = 1 NEWLINE
else:                           DEDENT DEDENT
    x = 2                   else :  NEWLINE INDENT ...
```

Depois de `x = 1` vêm dois DEDENT, que fecham o `if b` e depois o bloco do `if a`. Só então aparece o `else`, que só pode pertencer ao `if a`. A própria indentação decide o emparelhamento.

6 Exemplo trabalhado do início ao fim

Código-fonte:

```
def dobro(x: int) -> int:
    return x * 2
```

Tokens entregues pelo lexer:

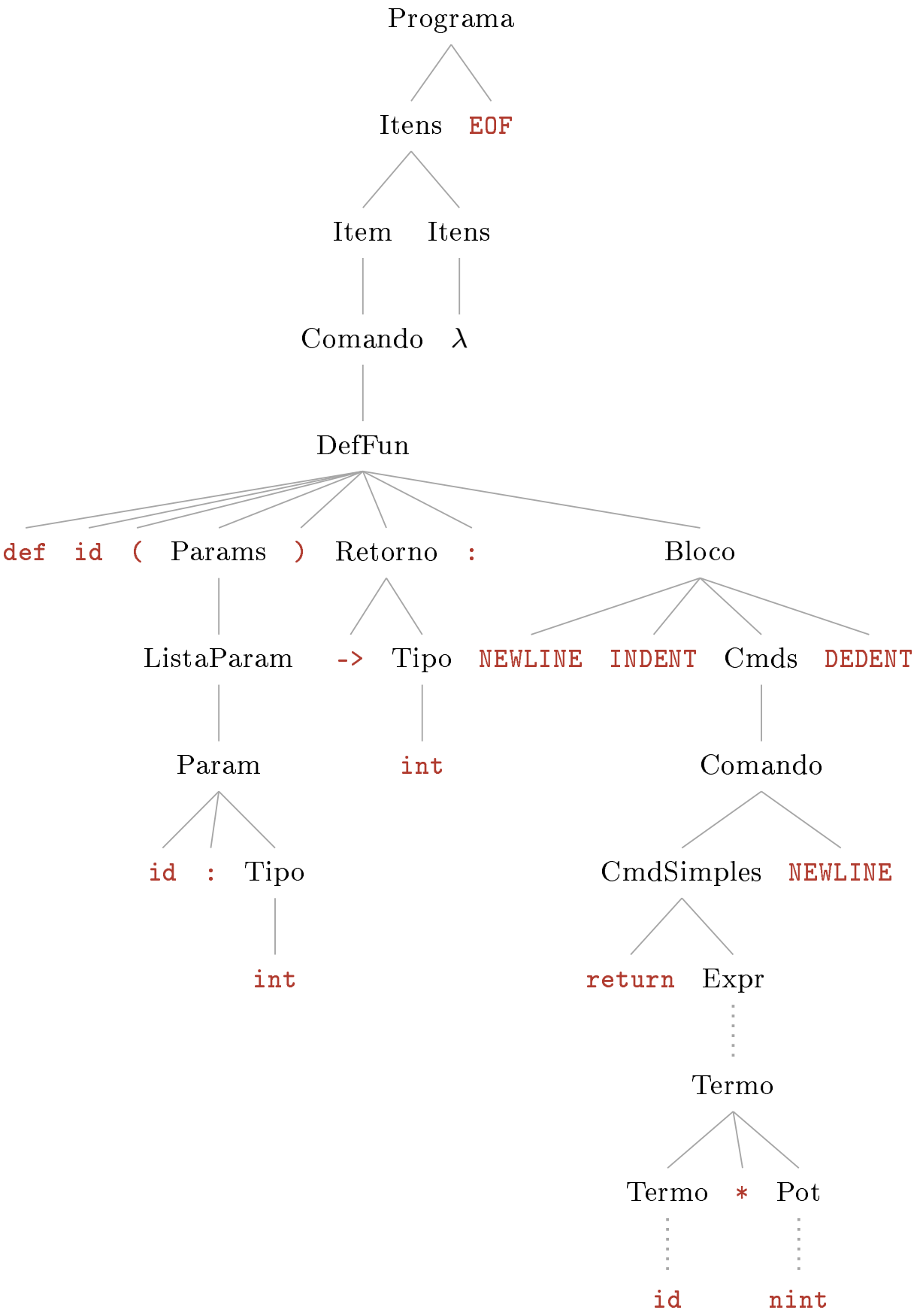
```
def id ( id : int ) -> int : NEWLINE INDENT
return id * nint NEWLINE DEDENT EOF
```

Derivação mais à esquerda ($\Rightarrow^*$ abrevia vários passos):

$$
\begin{aligned}
\textit{Programa} &\Rightarrow \textit{Itens}\ \texttt{EOF} \Rightarrow \textit{Item}\ \textit{Itens}\ \texttt{EOF} \Rightarrow \textit{Comando}\ \textit{Itens}\ \texttt{EOF} \\
&\Rightarrow \textit{DefFun}\ \textit{Itens}\ \texttt{EOF} \\
&\Rightarrow \texttt{def id (}\ \textit{Params}\ \texttt{)}\ \textit{Retorno}\ \texttt{:}\ \textit{Bloco}\ \textit{Itens}\ \texttt{EOF} \\
&\Rightarrow^* \texttt{def id (id : int) -> int :}\ \textit{Bloco}\ \textit{Itens}\ \texttt{EOF} \\
&\Rightarrow \cdots\ \texttt{: NEWLINE INDENT}\ \textit{Cmds}\ \texttt{DEDENT}\ \textit{Itens}\ \texttt{EOF} \\
&\Rightarrow^* \cdots\ \texttt{INDENT return}\ \textit{Expr}\ \texttt{NEWLINE DEDENT}\ \textit{Itens}\ \texttt{EOF} \\
&\Rightarrow^* \cdots\ \texttt{return id * nint NEWLINE DEDENT EOF}
\end{aligned}
$$

Na última linha, *Itens* virou λ (não há mais itens) e *Expr* desceu a escada até $\textit{Termo} \rightarrow \textit{Termo}\ \texttt{*}\ \textit{Pot}$.

Árvore de análise (com as cadeias abreviadas):



AST resultante (o que o parser deve produzir, sem pontuação e sem não terminais intermediários): uma definição de função chamada `dobro`, com parâmetro `x` do tipo `int`, retorno

`int`, e corpo com um `return` de uma multiplicação entre a variável `x` e o inteiro `2`. É assim que a apostila diferencia árvore de análise de AST.

7 O que a GLC não consegue dizer

Uma GLC só descreve a *forma* do programa, sem olhar o contexto. Algumas exigências do enunciado dependem do contexto:

Erro	GLC aceita?	Como tratar
`return` fora de função	sim	Flag "dentro de função" passada ao parser (no typed-peg, `%param`)
`break`/`continue` fora de laço	sim	Flag "dentro de laço"
`elif` sem `if`	não	Erro sintático automático
Vírgula dupla `[1,,2]`	não	Erro sintático automático
Falta de `:` depois do `if`	não	Erro sintático ("expected `':'`")
Tipos errados (`1 + 1.0`)	sim	Verificador de tipos (fase 3)
Variável não declarada	sim	Análise semântica

Em palavras simples

Pense na GLC como a "gramática do português": ela diz que "o gato comeu o peixe" está bem formada, mas não sabe se gatos comem peixe. As verificações de contexto ficam com o parser (flags) ou com a análise semântica.

8 Decisões de projeto (e o que você pode mudar)

Decisão	Motivo
`-` unário mais forte que `**`	Está escrito na especificação (§6.3)
`**` associa à direita	A especificação não diz; adotei o comportamento do Python 3, como o enunciado pede em caso de omissão
Comparação não associativa	Evita `bool < int`; casa com `%nonassoc` no Happy
`+=` etc. aceita qualquer *Alvo*	Mais permissivo que o §7.1, mas o Python aceita `xs[i] += 1`. Para restringir, troque por `id` *OpAtr Expr*
`else` do *Cond* recebe *Expr*	Permite `lambda` no ramo `else`, como no Python
Campos antes de métodos na sintaxe	A especificação exige essa ordem (§6.3)
DefFun dentro de *Comando*	Funções locais (regra 19) e `def` entre instruções de topo (exemplos 9.14 e 9.15)

9 Da GLC aos três parsers

A mesma gramática serve de base para os três analisadores pedidos no trabalho, com pequenas adaptações.

9.1 Descendente recursivo (megaparsec)

Cada não terminal vira uma função. A apostila (§6.2.6) mostra como lidar com a recursão à esquerda sem reescrever a gramática:

- Operadores associativos à esquerda ($E \to E \oplus T \mid T$) viram `chainl`. Vale para *Disj*, *Conj*, *Soma*, *Termo* e *Posfixo*.
- `**`, que é recursivo à direita, vira `chainr`.
- Listas separadas por vírgula (*ListaParam*, *ListaExpr*...) viram `sepBy` ou `sepBy1`.
- Repetições (*Itens*, *Cmds*, *Elifs*) viram `many` ou `some`.

Remoção formal da recursão à esquerda. Se você quiser provar que a gramática é adequada a um parser LL, aplique a transformação da apostila: $A \to A\alpha \mid \beta$ vira $A \to \beta A'$ e $A' \to \alpha A' \mid \lambda$. Para *Soma*, com $\alpha_1 =$ `+` *Termo*, $\alpha_2 =$ `-` *Termo* e $\beta =$ *Termo*:

$$
\begin{aligned}
\textit{Soma} &\to \textit{Termo}\ \textit{Soma'} \\
\textit{Soma'} &\to \texttt{+}\ \textit{Termo}\ \textit{Soma'} \\
&\mid\ \texttt{-}\ \textit{Termo}\ \textit{Soma'} \\
&\mid\ \lambda
\end{aligned}
$$

A linguagem é a mesma, mas a árvore deixa de mostrar a associatividade; o parser a recupera fazendo uma dobra à esquerda (`foldl`) sobre a lista de operandos, que é exatamente o que o `chainl` faz.

Prefixos em comum. Em *CmdSimples*, três alternativas começam com `id` (declaração, atribuição e expressão), o que é um conflito FIRST/FIRST. As saídas usuais são usar `try` no megaparsec, ou fatorar à esquerda: ler uma *Expr* e olhar o próximo token. Se for `:` (e a expressão for um `id`), é declaração; se for `=` ou um *OpAtr* (e a expressão for um *Alvo* válido), é atribuição; se for `NEWLINE`, é um comando-expressão.

9.2 PEG (typed-peg)

A escolha ordenada `/` resolve o prefixo em comum se você colocar as alternativas mais específicas primeiro: *Alvo* `=`, depois `id :`, depois *Expr* sozinha. Repetições usam `*` e `+`, e a recursão à esquerda também precisa virar iteração, como no `chainl`. Uma PEG não tem ambiguidade (a primeira alternativa que casa vence), mas isso torna a ordem das alternativas parte da gramática.

9.3 LALR (Happy)

Aqui a escada é substituída por *uma* única *Exp* ambígua, com a precedência declarada nas diretivas. Em Happy, o que é declarado depois tem precedência maior:

Ordem (menor → maior)	Diretiva	Operadores
1	`%left`	`or`
2	`%left`	`and`
3	`%right`	`not` (prefixo, com `%prec`)
4	`%nonassoc`	`== != < <= > >=`
5	`%left`	`+ -`
6	`%left`	`* / // %`
7	`%right`	`**`
8	`%right`	`NEG` (para o `-` unário, via `%prec NEG`)

O enunciado pede que cada conflito shift/reduce resolvido seja documentado no relatório. A tabela acima é a decisão global; os conflitos restantes (se houver) devem ser listados um a um.

10 Exercícios para estudar

1. Desenhe a árvore de análise de `a - b - c` (com cadeias abreviadas). De que lado ela aninha?
2. Por que `-2 ** 2` vale 4 nesta gramática? Qual regra é responsável?
3. `a < b < c` pertence à linguagem? Em qual regra a derivação trava?
4. Escreva os tokens (com `NEWLINE`, `INDENT` e `DEDENT`) do programa:
   `while n > 0:  n -= 1  print(n)`
5. Mostre que `x: int = 1 + 2` é derivado por *CmdSimples*. Qual alternativa de *CmdSimples* é usada?
6. Por que `f(x) = 3` é rejeitado e `xs[0] = 3` é aceito?
7. Rescreva *Termo* → *Termo* `*` *Pot* | *Pot* sem recursão à esquerda.

Respostas

1. A árvore tem *Soma* na raiz, com filhos *Soma* (que contém `a - b`), `-` e *Termo* (`c`). Aninha à esquerda: $(a - b) - c$.
2. A regra é *Pot* → *Unario* `**` *Pot*. O operando esquerdo de `**` é um *Unario*, que pode ser `-` *Unario*, então o `-` agrupa primeiro: $(-2)^2$.
3. Não. *Comp* → *Soma* *OpRel* *Soma* cobre `a < b`, e o segundo `<` não pode ser consumido por nenhuma regra a partir dali (depois de uma *Comp* só podem vir `and`, `or`, `if`, `:`, `)`, `,`, `NEWLINE` etc.), então há erro de sintaxe no segundo `<`.
4. `while id > nint :  NEWLINE INDENT id -= nint NEWLINE DEDENT id ( id ) NEWLINE EOF`
5. *CmdSimples* ⇒ `id :` *Tipo* `=` *Expr*, a quinta forma, a declaração com tipo. Depois *Tipo* ⇒ `int`, e *Expr* ⇒* *Soma* ⇒ *Soma* `+` *Termo* ⇒* `nint + nint`.
6. `f(x)` é um *Posfixo* que termina em `( ...)`, e *Alvo* só aceita `id`, *Posfixo* `[ ...]` ou *Posfixo* `. id`. Uma chamada não se encaixa em nenhum. Já `xs[0]` é *Posfixo* `[` *Expr* `]`, com *Posfixo* ⇒ *Primario* ⇒ `id`.
7. *Termo* → *Pot* *Termo'* e *Termo'* → `*` *Pot* *Termo'* | λ.

11 Resumo de uma página

- A GLC é a lista de regras que define quais sequências de tokens são programas válidos. O lexer fornece os terminais.
- `INDENT` e `DEDENT` fazem o papel das chaves, e por isso um bloco é `NEWLINE INDENT` *Cmds* `DEDENT`.
- Programa = lista de itens; item = classe ou comando; comando = simples, composto ou definição de função.
- Expressões formam uma escada: `lambda` < `if-else` < `or` < `and` < `not` < comparação < `+ -` < `* / // %` < `**` < `-` unário < chamadas, índices e campos < primários.
- Recursão à esquerda dá associatividade à esquerda, e recursão à direita dá associatividade à direita. Comparação não tem recursão, então não encadeia.
- *Alvo* é o que pode ficar à esquerda de `=`: `id`, `x[i]` ou `x.campo`.
- O que depende de contexto (`return` fora de função, `break` fora de laço, tipos) não é da GLC e fica com flags do parser ou com a análise semântica.
- Descendente recursivo → `chainl`/`chainr`/`many`; PEG → escolha ordenada; Happy → `%left`/`%right`/`%nonassoc`.